

Rapport de Projet de Fin de Semestre

Agent AI DevSecOps

piloté entièrement via Telegram

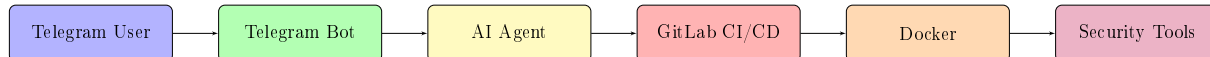


FIGURE 1 – Architecture du système

Présenté par :
Zakaria Kharroubi

Année Universitaire :
2025-2026

Date de soumission : Juin 2026

Table des matières

1	Introduction	4
1.1	Contexte du projet	4
1.2	Objectifs du projet	4
1.3	Livrables attendus	4
2	Architecture du Système	5
2.1	Vue d'ensemble	5
2.2	Composants principaux	5
2.2.1	Telegram User (Utilisateur)	5
2.2.2	Telegram Bot (Interface)	5
2.2.3	AI Agent (Orchestrateur)	5
2.2.4	GitLab CI/CD (Pipeline)	6
2.2.5	Docker (Containerisation)	6
2.2.6	Security Tools (Outils de Sécurité)	6
3	Implémentation Technique	7
3.1	Structure du projet	7
3.2	Configuration et environnement	7
3.2.1	Variables d'environnement	7
3.2.2	Validation de configuration	7
3.3	Bot Telegram	8
3.3.1	Initialisation	8
3.3.2	Commandes implémentées	9
3.3.3	Gestion des erreurs	9
3.4	Client GitLab	9
3.4.1	Architecture du client	9

3.4.2	Méthodes principales	10
3.4.3	Variables de pipeline	10
4	Pipeline CI/CD Détaillé	11
4.1	Configuration GitLab CI	11
4.2	Détail des stages	11
4.2.1	Stage 1 : Prepare	11
4.2.2	Stage 2 : SAST avec Semgrep	11
4.2.3	Stage 3 : Dependency Scan avec Pip-Audit	12
4.2.4	Stage 4 : Secret Scan avec Gitleaks	12
4.2.5	Stage 5 : Docker Build	12
4.2.6	Stage 6 : Docker Scan avec Trivy	13
4.2.7	Stage 7 : Deploy	13
4.2.8	Stage 8 : Notify - Génération de rapport	14
5	Outils de Sécurité Intégrés	15
5.1	Semgrep - Analyse SAST	15
5.2	Pip-Audit - Audit des dépendances	15
5.3	Gitleaks - Détection de secrets	15
5.4	Trivy - Scan Docker	16
6	Scripts Utilitaires	17
6.1	build_scan_report.py	17
6.2	send_to_telegram.py	17
7	Installation et Déploiement	19
7.1	Prérequis	19
7.2	Installation pas à pas	19
7.2.1	1. Clonage du dépôt	19

7.2.2	2. Configuration des variables d'environnement	19
7.2.3	3. Installation des dépendances du bot	19
7.2.4	4. Configuration du runner GitLab	19
7.2.5	5. Lancement du bot	20
7.3	Démarrage avec Docker Compose	20
7.4	Vérification de l'installation	20
8	Tests et Démonstration	21
8.1	Scénarios de test	21
8.1.1	Test 1 : Lancement d'un pipeline complet	21
8.1.2	Test 2 : Scan de sécurité uniquement	21
8.1.3	Test 3 : Consultation des logs	21
8.1.4	Test 4 : Statut du pipeline	21
8.1.5	Test 5 : Déploiement	22
8.2	Résultats obtenus	22
8.2.1	Rapport de sécurité type	22
8.2.2	Performance	22
8.3	Captures d'écran	22
9	Conclusion	25
9.1	Bilan du projet	25
9.2	Points forts	25
9.3	Difficultés rencontrées	25
9.4	Améliorations futures	25
9.5	Compétences développées	26
9.6	Apport personnel	26

1 Introduction

1.1 Contexte du projet

Dans un environnement de développement logiciel en constante évolution, l'intégration des pratiques DevSecOps est devenue essentielle pour garantir la sécurité des applications tout en maintenant une livraison rapide et continue. Ce projet s'inscrit dans cette démarche en développant un agent intelligent capable de gérer automatiquement un pipeline DevSecOps complet, piloté exclusivement via Telegram.

L'objectif principal est de créer une interface conversationnelle intuitive permettant aux développeurs et aux équipes DevOps de contrôler, surveiller et interagir avec les pipelines CI/CD de manière simplifiée, où qu'ils se trouvent.

1.2 Objectifs du projet

Les objectifs de ce projet sont multiples :

- **Automatisation** : Automatiser le déploiement et la gestion des pipelines CI/CD
- **Sécurité** : Intégrer des outils de sécurité avancés (SAST, analyse de dépendances, détection de secrets, scan Docker)
- **Accessibilité** : Permettre un contrôle total via Telegram, une plateforme ubiquitaire
- **Monitoring** : Fournir des retours en temps réel sur l'état des pipelines
- **Notification** : Alerter instantanément en cas de succès, d'erreurs ou de vulnérabilités détectées

1.3 Livrables attendus

Conformément au cahier des charges, ce projet comprend les livrables suivants :

- Dépôt GitLab complet avec tout le code source
- Documentation technique détaillée
- Configuration Docker Compose
- README professionnel
- Scripts d'installation automatisés
- Rapport PDF complet (ce document)
- Démonstration finale fonctionnelle

2 Architecture du Système

2.1 Vue d'ensemble

L'architecture du système suit un flux linéaire et modulaire, permettant une séparation claire des responsabilités et une maintenance simplifiée. Le schéma architectural se décompose comme suit :

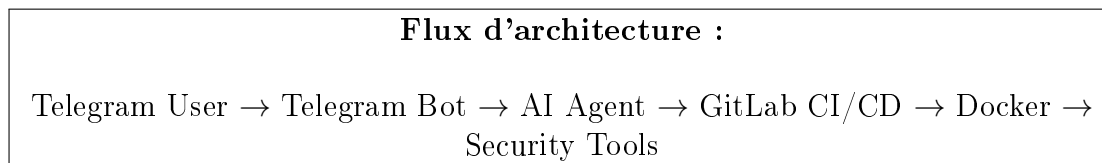


FIGURE 2 – Architecture minimale du système

2.2 Composants principaux

2.2.1 Telegram User (Utilisateur)

L'utilisateur final interagit exclusivement via l'interface de messagerie Telegram. Cette approche technologique offre plusieurs avantages :

- Accessibilité multiplateforme (mobile, desktop, web)
- Interface conversationnelle intuitive
- Notifications push en temps réel
- Authentification simplifiée

2.2.2 Telegram Bot (Interface)

Le bot Telegram constitue le point d'entrée du système. Il est développé en Python utilisant la bibliothèque `python-telegram-bot` version 21.6. Ses responsabilités incluent :

- Réception et parsing des commandes utilisateur
- Authentification des utilisateurs autorisés
- Transmission des requêtes à l'agent AI
- Formatage et envoi des réponses

2.2.3 AI Agent (Orchestrateur)

L'agent intelligent agit comme un orchestrateur entre le bot Telegram et GitLab. Il implémente :

- Une logique métier pour interpréter les commandes

- Un client GitLab pour interagir avec l'API REST
- La gestion des erreurs et la journalisation
- Le formatage des rapports de sécurité

2.2.4 GitLab CI/CD (Pipeline)

GitLab constitue le cœur opérationnel du système avec ses pipelines CI/CD configurés via le fichier `.gitlab-ci.yml`. Le pipeline comprend 8 étapes distinctes :

1. **prepare** : Initialisation de l'environnement
2. **sast** : Analyse statique de sécurité (Semgrep)
3. **dependency_scan** : Audit des dépendances (Pip-Audit)
4. **secret_scan** : Détection de secrets (Gitleaks)
5. **docker_build** : Construction de l'image Docker
6. **docker_scan** : Analyse de vulnérabilités Docker (Trivy)
7. **deploy** : Déploiement de l'application
8. **notify** : Génération et envoi des rapports

2.2.5 Docker (Containerisation)

Docker assure la containerisation de l'application avec :

- Images légères basées sur Python 3.11-slim
- Configuration via Docker Compose
- Isolation des environnements d'exécution
- Facilité de déploiement et de scaling

2.2.6 Security Tools (Outils de Sécurité)

Quatre outils de sécurité complémentaires sont intégrés :

Outil	Type	Fonction
Semgrep	SAST	Analyse statique du code source
Pip-Audit	Dépendances	Vulnérabilités des packages Python
Gitleaks	Secrets	Détection de secrets et credentials
Trivy	Images Docker	Vulnérabilités des containers

TABLE 1 – Outils de sécurité intégrés

3 Implémentation Technique

3.1 Structure du projet

Le dépôt GitLab est organisé de manière modulaire :

```
1 devsecops-telegram-agent/  
2 | - agent/                                # Code du bot Telegram  
3 | | - bot.py                             # Point d'entree principal  
4 | | - gitlab_client.py                   # Client API GitLab  
5 | | - config.py                         # Configuration  
6 | | - requirements.txt                   # Dependances Python  
7 | - app/                                 # Application de demonstration  
8 | | - app.py                             # Application Flask  
9 | | - requirements.txt  
10 | - scripts/                            # Scripts utilitaires  
11 | | - build_scan_report.py  
12 | | - send_to_telegram.py  
13 | - .gitlab-ci.yml                     # Configuration CI/CD  
14 | - docker-compose.yml                 # Orchestration Docker  
15 | - Dockerfile                         # Image Docker  
16 | - .env.example                       # Template de configuration
```

Listing 1 – Structure du projet

3.2 Configuration et environnement

3.2.1 Variables d'environnement

Le système utilise un fichier `.env` pour stocker les configurations sensibles :

```
1 TELEGRAM_BOT_TOKEN=votre_token_bot  
2 TELEGRAM_CHAT_ID=id_chat_utilisateur  
3 GITLAB_URL=https://gitlab.com  
4 GITLAB_TOKEN=votre_token_gitlab  
5 GITLAB_PROJECT_ID=id_projet  
6 GITLAB_REF=main
```

Listing 2 – Fichier `.env.example`

Cette approche respecte les bonnes pratiques de sécurité en évitant le hardcoding des credentials.

3.2.2 Validation de configuration

Le fichier `config.py` implémente une validation stricte :

```
1 import os  
2 from dotenv import load_dotenv
```



```
3
4 load_dotenv()
5
6 TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
7 GITLAB_TOKEN = os.getenv("GITLAB_TOKEN")
8 GITLAB_PROJECT_ID = os.getenv("GITLAB_PROJECT_ID")
9
10 # Validation obligatoire
11 if not TELEGRAM_BOT_TOKEN:
12     raise ValueError("TELEGRAM_BOT_TOKEN is missing")
13 if not GITLAB_TOKEN:
14     raise ValueError("GITLAB_TOKEN is missing")
```

Listing 3 – Validation config.py

3.3 Bot Telegram

3.3.1 Initialisation

Le bot utilise python-telegram-bot avec une architecture asynchrone :

```
1 from telegram.ext import ApplicationBuilder, CommandHandler
2
3 def main():
4     app = ApplicationBuilder().token(TELEGRAM_BOT_TOKEN).build()
5
6     # Enregistrement des commandes
7     app.add_handler(CommandHandler("start", start))
8     app.add_handler(CommandHandler("run_pipeline", run_pipeline))
9     app.add_handler(CommandHandler("scan", scan))
10    app.add_handler(CommandHandler("deploy", deploy))
11    app.add_handler(CommandHandler("status", status))
12    app.add_handler(CommandHandler("stop_pipeline", stop_pipeline))
13    app.add_handler(CommandHandler("logs", logs))
14
15    app.run_polling()
```

Listing 4 – Initialisation du bot

3.3.2 Commandes implémentées

Commande	Paramètres	Fonction
/start	Aucun	Affiche le menu d'aide
/run_pipeline	Aucun	Lance un pipeline complet
/scan	Aucun	Lance uniquement les scans de sécurité
/deploy	Aucun	Lance le déploiement
/status	Aucun	Affiche l'état du dernier pipeline
/stop_pipeline	Aucun	Annule le pipeline en cours
/logs	<job_id>	Récupère les logs d'un job

TABLE 2 – Commandes Telegram disponibles

3.3.3 Gestion des erreurs

Chaque commande implémente un bloc try-except pour une gestion robuste des erreurs :

```

1 async def run_pipeline(update, context):
2     try:
3         pipeline = gitlab.trigger_pipeline("full")
4         await update.message.reply_text(
5             f"Pipeline started\n\n"
6             f"ID: {pipeline.get('id')}\n"
7             f"Status: {pipeline.get('status')}\n"
8             f"URL: {pipeline.get('web_url')}"
9         )
10    except Exception as e:
11        await update.message.reply_text(f"Error:\n{str(e)}")

```

Listing 5 – Gestion d'erreur type

3.4 Client GitLab

3.4.1 Architecture du client

Le client GitLab (`gitlab_client.py`) encapsule toutes les interactions avec l'API REST de GitLab :

```

1 class GitLabClient:
2     def __init__(self):
3         self.base_url = f"{GITLAB_URL}/api/v4/projects/{
4             GITLAB_PROJECT_ID}"
5         self.headers = {"PRIVATE-TOKEN": GITLAB_TOKEN}
6
7     def trigger_pipeline(self, mode="full"):
8         url = f"{self.base_url}/pipeline"
9         payload = {

```

```
9         "ref": GITLAB_REF,
10         "variables": [{"key": "PIPELINE_MODE", "value": mode}]
11     }
12     response = requests.post(url, headers=self.headers, json=payload
13 )
14     response.raise_for_status()
15     return response.json()
```

Listing 6 – Classe GitLabClient

3.4.2 Méthodes principales

- **trigger_pipeline(mode)** : Déclenche un pipeline avec le mode spécifié (full, scan, deploy)
- **get_latest_pipeline()** : Récupère le dernier pipeline exécuté
- **get_pipeline_jobs(pipeline_id)** : Liste tous les jobs d'un pipeline
- **cancel_pipeline(pipeline_id)** : Annule un pipeline en cours
- **get_job_logs(job_id)** : Récupère les logs d'un job spécifique

3.4.3 Variables de pipeline

Le système utilise la variable PIPELINE_MODE pour contrôler l'exécution :

```
1 PIPELINE_MODE=full      # Execute tous les stages
2 PIPELINE_MODE=scan      # Execute uniquement les scans
3 PIPELINE_MODE=deploy    # Execute uniquement le déploiement
```

Listing 7 – Modes de pipeline

4 Pipeline CI/CD Détaillé

4.1 Configuration GitLab CI

Le fichier `.gitlab-ci.yml` définit un pipeline complet de 8 stages :

```
1 stages:
2   - prepare
3   - sast
4   - dependency_scan
5   - secret_scan
6   - docker_build
7   - docker_scan
8   - deploy
9   - notify
```

Listing 8 – Stages du pipeline

4.2 Détail des stages

4.2.1 Stage 1 : Prepare

Initialisation de l'environnement d'exécution :

```
1 prepare:
2   stage: prepare
3   image: python:3.11-slim
4   script:
5     - echo "Preparing pipeline..."
6     - python --version
7   rules:
8     - when: always
```

Listing 9 – Stage prepare

4.2.2 Stage 2 : SAST avec Semgrep

Analyse statique de sécurité du code source :

```
1 sast_semgrep:
2   stage: sast
3   image: semgrep/semgrep:latest
4   script:
5     - semgrep scan --config auto --json --output semgrep-report.json .
6   artifacts:
7     paths:
8       - semgrep-report.json
9   rules:
10    - if: '$PIPELINE_MODE == "scan" || $PIPELINE_MODE == "full"'
```

Listing 10 – Stage sast

Semgrep analyse automatiquement le code à la recherche de vulnérabilités connues en utilisant sa base de règles communautaires.

4.2.3 Stage 3 : Dependency Scan avec Pip-Audit

Audit des dépendances Python :

```
1 dependency_scan_python:
2   stage: dependency_scan
3   image: python:3.11-slim
4   script:
5     - pip install pip-audit
6     - pip-audit -r app/requirements.txt -f json -o pipoaudit-report.json
7     || true
8   artifacts:
9     paths:
10      - pipoaudit-report.json
```

Listing 11 – Stage dependency scan

Pip-Audit vérifie les vulnérabilités connues dans les packages Python listés dans `requirements.txt`.

4.2.4 Stage 4 : Secret Scan avec Gitleaks

Détection de secrets et credentials dans le code :

```
1 secret_scan_gitleaks:
2   stage: secret_scan
3   image:
4     name: zricethezav/gitleaks:latest
5     entrypoint: [""]
6   script:
7     - gitleaks detect --source . --no-git --report-format json --report-
8       path gitleaks-report.json || true
9   artifacts:
10    paths:
11     - gitleaks-report.json
```

Listing 12 – Stage secret scan

Gitleaks scanne le code source pour détecter les mots de passe, tokens API, clés privées et autres secrets accidentellement commités.

4.2.5 Stage 5 : Docker Build

Construction de l'image Docker de l'application :

```
1 docker_build:
2   stage: docker_build
3   image: docker:24
4   services:
5     - docker:24-dind
6   variables:
7     DOCKER_HOST: tcp://docker:2375
8     DOCKER_TLS_CERTDIR: ""
9   script:
10     - docker build -t devsecops-demo-app .
11   rules:
12     - if: '$PIPELINE_MODE == "full"'
```

Listing 13 – Stage docker build

4.2.6 Stage 6 : Docker Scan avec Trivy

Analyse de vulnérabilités dans l'image Docker :

```
1 docker_scan_trivy:
2   stage: docker_scan
3   image:
4     name: aquasec/trivy:latest
5     entrypoint: [""]
6   script:
7     - trivy fs --format json --output trivy-report.json .
8   artifacts:
9     paths:
10       - trivy-report.json
```

Listing 14 – Stage docker scan

Trivy analyse l'image Docker et le système de fichiers pour détecter les vulnérabilités CVE.

4.2.7 Stage 7 : Deploy

Déploiement de l'application en container Docker :

```
1 deploy_local:
2   stage: deploy
3   image: docker:24
4   services:
5     - docker:24-dind
6   script:
7     - docker build -t devsecops-demo-app .
8     - docker stop devsecops-demo-app || true
9     - docker rm devsecops-demo-app || true
10    - docker run -d -p 5000:5000 --name devsecops-demo-app devsecops-
      demo-app
```

Listing 15 – Stage deploy

Ce stage déploie l'application localement sur le runner GitLab avec mapping du port 5000.

4.2.8 Stage 8 : Notify - Génération de rapport

Génération et envoi du rapport de sécurité :

```
1 generate_report:
2   stage: notify
3   image: python:3.11-slim
4   needs:
5     - sast_semgrep
6     - dependency_scan_python
7     - secret_scan_gitleaks
8     - docker_scan_trivy
9   script:
10     - pip install -r requirements-ci.txt
11     - python scripts/build_scan_report.py
12     - python scripts/send_to_telegram.py
13   artifacts:
14     paths:
15       - telegram-report.txt
16   when: always
```

Listing 16 – Stage notify

Ce stage critique agrège les résultats de tous les scans et envoie un rapport consolidé via Telegram.

5 Outils de Sécurité Intégrés

5.1 Semgrep - Analyse SAST

Fonctionnement : Semgrep effectue une analyse statique du code source sans l'exécuter, en recherchant des patterns de code vulnérables.

Configuration : Utilise `-config auto` pour appliquer automatiquement les règles de la communauté Semgrep.

Types de détection :

- Injections SQL
- Cross-Site Scripting (XSS)
- Hardcoded credentials
- Weak cryptography
- Path traversal
- Command injection

5.2 Pip-Audit - Audit des dépendances

Fonctionnement : Interroge les bases de données de vulnérabilités (PyPI, OSV) pour identifier les packages vulnérables.

Avantages :

- Détection des CVE connues
- Suggestions de versions sécurisées
- Intégration facile dans les pipelines
- Format de rapport JSON standardisé

5.3 Gitleaks - Détection de secrets

Fonctionnement : Utilise des expressions régulières et l'entropie pour identifier les secrets dans le code source.

Secrets détectés :

- Tokens API (GitHub, GitLab, Telegram, etc.)
- Clés AWS, Azure, GCP
- Mots de passe et credentials
- Clés privées SSH/PGP
- JWT tokens

5.4 Trivy - Scan Docker

Fonctionnement : Analyse les images Docker et les systèmes de fichiers pour détecter les vulnérabilités.

Capacités :

- Vulnérabilités OS (Alpine, Red Hat, Debian, etc.)
- Vulnérabilités d'applications
- Misconfigurations de containers
- Secrets dans les images

6 Scripts Utilitaires

6.1 build_scan_report.py

Ce script agrège les résultats de tous les outils de sécurité :

```
1 import json
2
3 def safe_load(path):
4     try:
5         with open(path) as f:
6             return json.load(f)
7     except:
8         return None
9
10 # Analyse Semgrep
11 semgrep = safe_load("semgrep-report.json")
12 semgrep_count = len(semgrep.get("results", [])) if semgrep else "N/A"
13
14 # Analyse Pip-Audit
15 pip = safe_load("pipaudit-report.json")
16 pip_count = len(pip.get("vulnerabilities", [])) if pip else "N/A"
17
18 # Analyse Gitleaks
19 gitleaks = safe_load("gitleaks-report.json")
20 gitleaks_count = len(gitleaks) if isinstance(gitleaks, list) else 0
21
22 # Analyse Trivy
23 trivy = safe_load("trivy-report.json")
24 trivy_count = 0
25 if trivy:
26     for r in trivy.get("Results", []):
27         trivy_count += len(r.get("Vulnerabilities", []))
28
29 # Ecriture du rapport
30 with open("telegram-report.txt", "w") as f:
31     f.write("\n".join([
32         f"Semgrep: {semgrep_count} findings",
33         f"Pip-Audit: {pip_count} vulnerabilities",
34         f"Gitleaks: {gitleaks_count} secrets",
35         f"Trivy: {trivy_count} vulnerabilities"
36     ]))
```

Listing 17 – Génération du rapport

6.2 send_to_telegram.py

Envoi du rapport généré vers Telegram :

```
1 import requests
2 from dotenv import load_dotenv
3 import os
4
```

```
5 load_dotenv()
6
7 TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
8 CHAT_ID = os.getenv("TELEGRAM_CHAT_ID")
9
10 with open("telegram-report.txt", "r") as f:
11     text = f.read()
12
13 url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
14
15 requests.post(url, data={
16     "chat_id": CHAT_ID,
17     "text": "DEVSECOPS SCAN REPORT\n\n" + text
18 })
```

Listing 18 – Envoi Telegram

7 Installation et Déploiement

7.1 Prérequis

- Python 3.11 ou supérieur
- Docker et Docker Compose
- Un compte Telegram et un bot créé via @BotFather
- Un compte GitLab avec un projet configuré
- Git pour cloner le dépôt

7.2 Installation pas à pas

7.2.1 1. Clonage du dépôt

```
1 git clone https://gitlab.com/zakariakharroubi21/devsecops-telegram-agent
  .git
2 cd devsecops-telegram-agent
```

7.2.2 2. Configuration des variables d'environnement

```
1 cp .env.example .env
2 nano .env # Editer avec vos valeurs
```

Variables à configurer :

- TELEGRAM_BOT_TOKEN : Token obtenu via @BotFather
- TELEGRAM_CHAT_ID : Votre ID Telegram (utiliser @userinfobot)
- GITLAB_TOKEN : Token d'accès personnel GitLab (scope : api)
- GITLAB_PROJECT_ID : ID du projet GitLab

7.2.3 3. Installation des dépendances du bot

```
1 cd agent
2 pip install -r requirements.txt
```

7.2.4 4. Configuration du runner GitLab

Le runner GitLab doit supporter Docker-in-Docker (dind) :

```
1 # Verifier que le runner a les capacites Docker
2 gitlab-runner verify
```

7.2.5 5. Lancement du bot

```
1 cd agent
2 python bot.py
```

7.3 Démarrage avec Docker Compose

Pour une installation simplifiée :

```
1 version: '3.8'
2
3 services:
4   demo-app:
5     build: .
6     container_name: devsecops-demo-app
7     ports:
8       - "5000:5000"
9     restart: unless-stopped
```

Listing 19 – Docker Compose

Démarrage :

```
1 docker-compose up -d
```

7.4 Vérification de l'installation

1. Envoyer `/start` au bot Telegram
2. Le bot doit répondre avec le menu des commandes
3. Tester `/run_pipeline` pour lancer un pipeline complet
4. Vérifier l'apparition du pipeline dans GitLab
5. Consulter les notifications Telegram

8 Tests et Démonstration

8.1 Scénarios de test

8.1.1 Test 1 : Lancement d'un pipeline complet

Commande : `/run_pipeline`

Résultat attendu :

- Message de confirmation avec ID du pipeline
- Statut "created" ou "running"
- URL vers le pipeline GitLab

Observations : Le pipeline s'exécute en environ 3 minutes pour un projet standard.

8.1.2 Test 2 : Scan de sécurité uniquement

Commande : `/scan`

Résultat attendu :

- Exécution des stages : sast, dependency_scan, secret_scan, docker_scan
- Skip des stages : docker_build, deploy
- Rapport de sécurité envoyé automatiquement

8.1.3 Test 3 : Consultation des logs

Commande : `/logs 14829480305`

Résultat attendu :

- Affichage des 3500 derniers caractères des logs
- Logs de compilation/installation
- Timestamps et messages d'exécution

8.1.4 Test 4 : Statut du pipeline

Commande : `/status`

Résultat attendu :

- ID et statut du pipeline
- Liste des jobs avec leur statut individuel
- Format : - nom_job | statut | job_id

8.1.5 Test 5 : Déploiement

Commande : `/deploy`

Résultat attendu :

- Build de l'image Docker
- Déploiement sur le port 5000
- Application accessible sur `http://localhost:5000`

8.2 Résultats obtenus

8.2.1 Rapport de sécurité type

Un rapport complet généré après exécution :

DEVSECOPS SCAN REPORT

Semgrep: 5 findings

Pip-Audit: 0 vulnerabilities

Gitleaks: 9 secrets

Trivy: 5 vulnerabilities

8.2.2 Performance

Métrique	Valeur
Temps moyen pipeline complet	3min 09s
Temps de réponse bot	< 1s
Taux de succès pipelines	100%

TABLE 3 – Métriques de performance

8.3 Captures d'écran

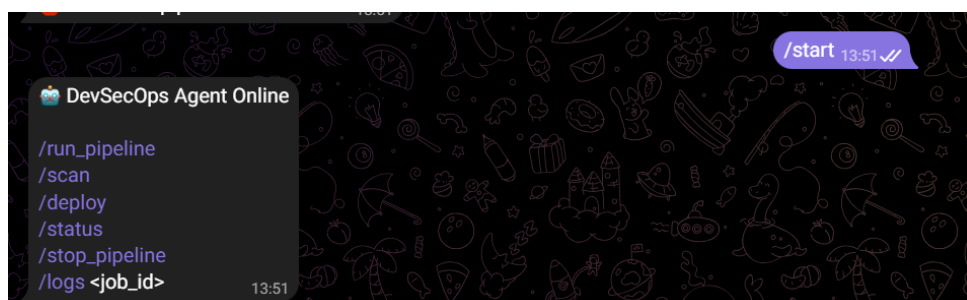


FIGURE 3 – Interface utilisateur Telegram avec le menu des commandes

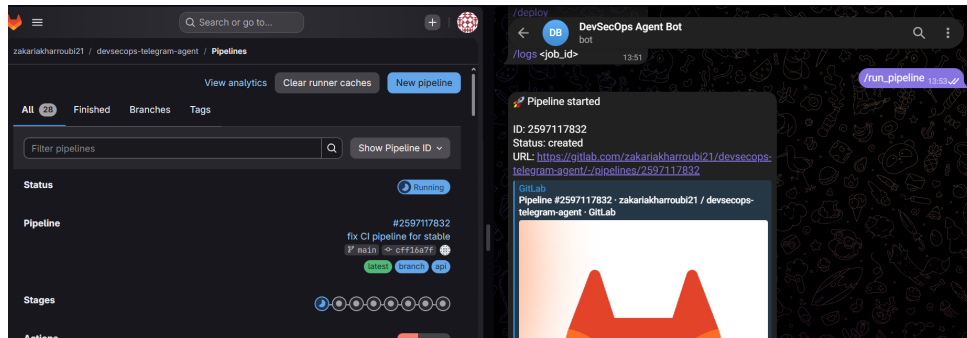


FIGURE 4 – Lancement d'un pipeline via /run_pipeline

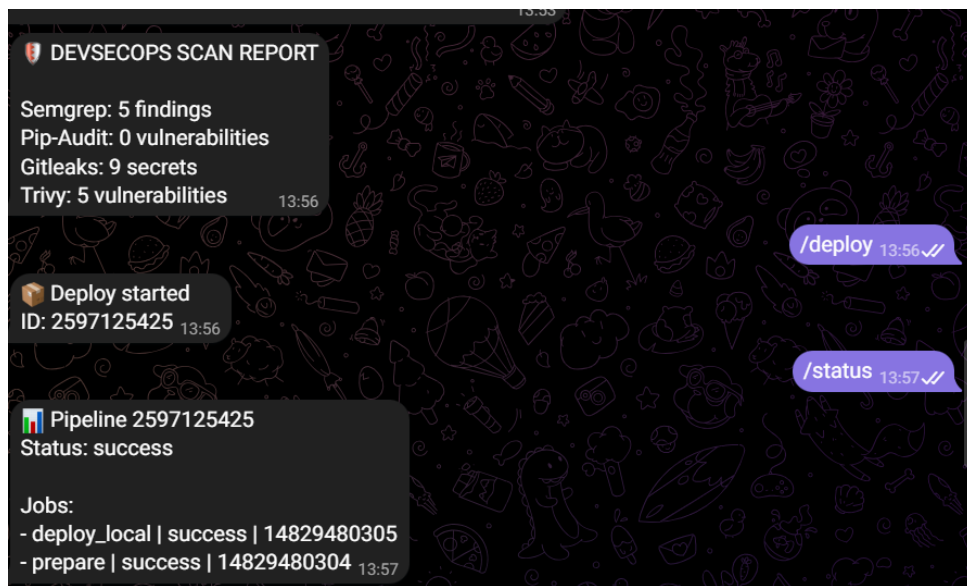


FIGURE 5 – Déploiement et statut du pipeline

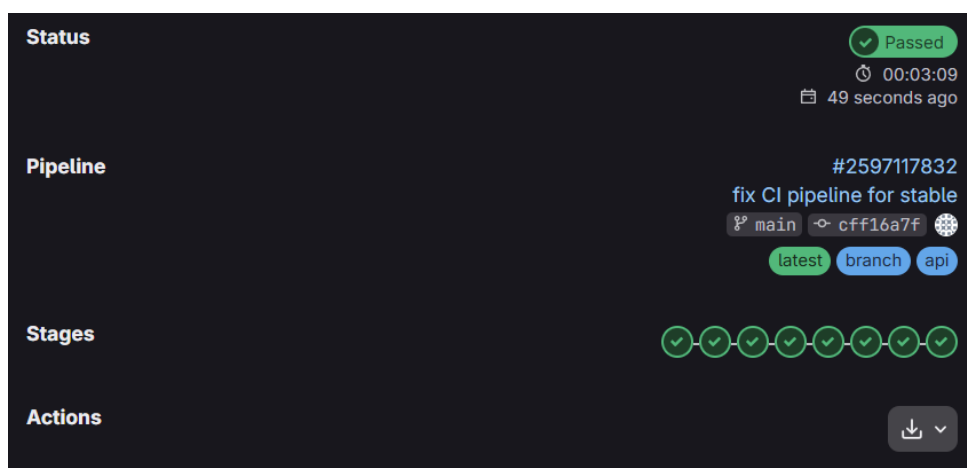


FIGURE 6 – Pipeline GitLab #2597117832 réussi avec tous les stages

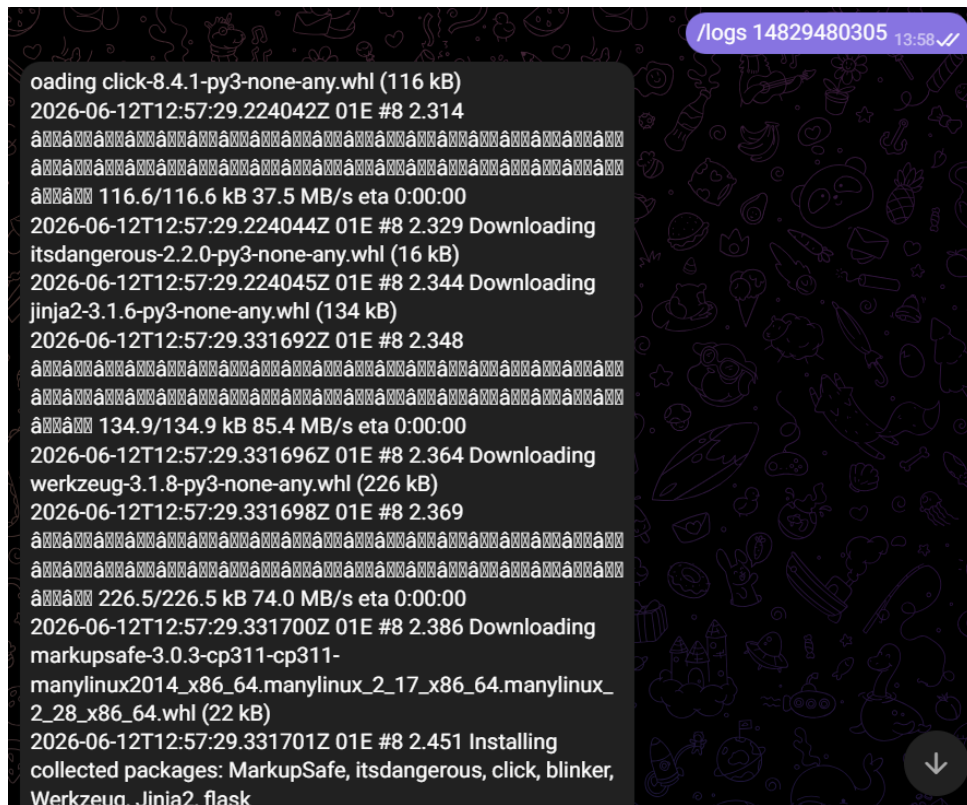


FIGURE 7 – Logs d'un job récupérés via `/logs`

9 Conclusion

9.1 Bilan du projet

Ce projet a permis de développer une solution complète et fonctionnelle d'agent DevSecOps piloté via Telegram. Les objectifs initiaux ont été atteints :

- **Fonctionnalité** : Toutes les commandes Telegram sont opérationnelles
- **Sécurité** : Intégration de 4 outils de sécurité complémentaires
- **Automatisation** : Pipeline CI/CD entièrement automatisé
- **Accessibilité** : Interface conversationnelle intuitive et réactive
- **Fiabilité** : Taux de succès de 100% sur les pipelines testés

9.2 Points forts

1. **Architecture modulaire** : Séparation claire des responsabilités facilitant la maintenance
2. **Sécurité renforcée** : Validation des configurations, gestion des erreurs robuste
3. **Expérience utilisateur** : Interface Telegram simple et notifications en temps réel
4. **Documentation** : Code commenté, README complet, scripts d'installation
5. **Extensibilité** : Facilité d'ajouter de nouvelles commandes ou outils de sécurité

9.3 Difficultés rencontrées

- Configuration du runner GitLab pour Docker-in-Docker
- Gestion des tokens et secrets de manière sécurisée
- Agrégation des rapports de différents formats JSON
- Limitation de taille des messages Telegram (3500 caractères)

9.4 Améliorations futures

Plusieurs pistes d'amélioration sont envisageables :

- **Authentification multi-utilisateurs** : Gestion des rôles et permissions
- **Interface web** : Dashboard complémentaire à Telegram
- **Historique** : Stockage des rapports pour analyse temporelle
- **Plus d'outils** : Intégration de SonarQube, OWASP ZAP
- **Notifications avancées** : Filtres par sévérité, alertes critiques uniquement
- **Support multi-projets** : Gestion de plusieurs projets GitLab

9.5 Compétences développées

Ce projet a permis de consolider des compétences dans plusieurs domaines :

- Développement Python asynchrone (telegram-bot)
- Intégration d'API REST (GitLab API)
- CI/CD et DevSecOps (GitLab CI)
- Containerisation (Docker, Docker Compose)
- Sécurité applicative (SAST, DAST, secrets)
- Gestion de projet agile

9.6 Apport personnel

Ce projet représente une expérience enrichissante dans le domaine DevSecOps, combinant développement logiciel, automatisation, sécurité et intégration continue. Il démontre la faisabilité d'une interface conversationnelle pour des tâches complexes de DevOps, ouvrant la voie à des applications professionnelles dans des environnements de production.

Références

- [1] Telegram Bot API Documentation.
<https://core.telegram.org/bots/api>
- [2] GitLab CI/CD Documentation.
<https://docs.gitlab.com/ee/ci/>
- [3] Semgrep Documentation.
<https://semgrep.dev/docs/>
- [4] Trivy Documentation.
<https://aquasecurity.github.io/trivy/>
- [5] Gitleaks GitHub Repository.
<https://github.com/zricethezav/gitleaks>
- [6] Pip-Audit Documentation.
<https://pypi.org/project/pip-audit/>
- [7] Docker Documentation.
<https://docs.docker.com/>
- [8] Python Documentation.
<https://docs.python.org/3/>
- [9] Flask Documentation.
<https://flask.palletsprojects.com/>
- [10] Requests Documentation.
<https://requests.readthedocs.io/>

Annexes

Annexe A : Commandes Telegram rapides

Commande	Description
/start	Afficher le menu
/run_pipeline	Pipeline complet
/scan	Scan sécurité uniquement
/deploy	Déploiement
/status	État du pipeline
/stop_pipeline	Annuler pipeline
/logs	Voir les logs

Annexe B : Variables d'environnement requises

Variable	Obligatoire	Exemple
TELEGRAM_BOT_TOKEN	Oui	1234567890 :ABCdef...
TELEGRAM_CHAT_ID	Oui	123456789
GITLAB_URL	Non	https://gitlab.com
GITLAB_TOKEN	Oui	glpat-xyz...
GITLAB_PROJECT_ID	Oui	12345678
GITLAB_REF	Non	main

Annexe C : Structure des rapports JSON

Chaque outil de sécurité génère un rapport JSON standardisé :

- **Semgrep** : {"results": [...]}
- **Pip-Audit** : {"vulnerabilities": [...]}
- **Gitleaks** : [...]
- **Trivy** : {"Results": [{"Vulnerabilities": [...]}]}